

CSE 4102: Computer Graphics and Image Processing Laboratory

**SENTINEL: A FULLY PROCEDURAL GAME ENGINE
AND OPEN-WORLD MILITARY SHOOTER**

By

Md. Shifat Hasan

Roll: 2107067



**Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
October 8, 2026**

Sentinel: A Fully Procedural Game Engine and Open-World Military Shooter

By

Md. Shifat Hasan

Roll: 2107067

Department: Computer Science and Engineering

Course: CSE 4102, Computer Graphics and Image Processing Laboratory

Year and Term: 4th Year 1st Term

Instructors:

Md Tajmilur Rahman

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Md. Mubtashim Abrar Nihal

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Signature

Signature

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

October 8, 2026

Acknowledgment

All praise belongs to the Almighty Allah, whose blessing and mercy allowed me to complete this project. I am sincerely grateful to my course instructors, Md Tajmilur Rahman and Md. Muqtashim Abrar Nihal, for their guidance, their patient feedback during the laboratory sessions and the freedom to choose an ambitious topic. I thank the Department of Computer Science and Engineering of Khulna University of Engineering & Technology for the laboratory environment in which this work was carried out.

I also thank the open-source communities behind the Odin programming language, GLFW and miniaudio, and the authors of the papers and teaching notes listed in the references: this project stands on their published work. Finally, I thank my family and friends for their support and for patiently playtesting the game.

Author

Abstract

Most video games ship with gigabytes of files made by artists: 3D models, photographs for surfaces, recorded sounds. *Sentinel* takes the opposite route. It is a complete game engine and an open-world military shooter in which *nothing* is loaded from a model, image or sound file. Every tree, building, soldier, vehicle, surface texture, cloud and sound effect is created while the program starts, from short mathematical recipes and a random seed. Only the small text files that hold the shader programs are read from disk.

The engine is written in the Odin language on OpenGL 4.1 (the newest version macOS offers, which has no compute shaders), and is organised in six layers, each allowed to depend only on the layers beneath it. A deferred renderer draws the scene in two steps: first it records what every pixel is made of, then it lights it with a sun and moon, spot and point lamps, cascaded shadow maps, screen-space ambient occlusion and reflections, an atmospheric sky with wind-driven clouds, fog, bloom, tone mapping, colour grading and anti-aliasing. Surfaces come from 18 texture recipes built on periodic gradient noise, domain warping and cellular noise; normal maps are derived from height fields with a Scharr filter. Objects and characters are tables of primitives with noise deformers. On top sit collision, a ladder state machine, four ground vehicles, a helicopter, enemy soldiers with stealth rules, synthesised audio and a five-objective rescue mission in the spirit of *Project I.G.I.*

The project was measured and verified, not just drawn. At 1080p the renderer needs about 10.9 ms of GPU time per frame; 281 unit tests, 30 GL checks, 8 318 texture checks (including exact tiling of every material), 318 567 renderer checks and two scripted full playthroughs pass. The report explains each technique in plain language, supports it with the governing equations, and documents a debugging case in which a star in the fog colour painted a bright vertical line across the night scene.

Contents

	PAGE
Title Page	i
Acknowledgment	ii
Abstract	iii
Contents	iv
List of Tables and Figures	v
CHAPTER I Introduction	1
1.1 Introduction	1
1.2 Background and Problem Statement	1
1.3 Objectives	1
1.4 Scope	1
1.5 Unfamiliarity of the Problem and its Solution	2
1.6 Project Planning and Work Distribution	2
1.7 Applications of the Work	2
1.8 Organization of the Project	3
CHAPTER II Related Works	3
2.1 Introduction	3
2.2 Related Works	3
2.3 Discussion	3
CHAPTER III Methodology	3
3.1 Introduction	3
3.2 Overall Design	4
3.3 Procedural Geometry and Characters	4
3.4 Procedural Textures	5
3.5 The Rendering Pipeline	6
3.6 Sky, Time of Day and Clouds	7
3.7 World, Physics and Gameplay	7
3.8 Conclusion	8
CHAPTER IV Implementation, Results and Discussions	8
4.1 Introduction	8
4.2 Experimental Setup	8
4.3 Evaluation Metrics	9
4.4 Implementation and Results	9
4.5 Objectives Achieved	10
4.6 Conclusion	11
CHAPTER V Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues	11
5.1 Intellectual Property Considerations	11
5.2 Ethical Considerations	11
5.3 Safety Considerations	11
5.4 Legal Considerations	11
5.5 Impact on Societal, Health and Cultural Issues	12
5.6 Impact on the Environment and Sustainability	12
CHAPTER VI Addressing Complex Engineering Problems and Activities	12
6.1 Complex Engineering Problems	12
6.2 Complex Engineering Activities	12
CHAPTER VII Conclusions	12
7.1 Summary	12
7.2 Limitations	13
7.3 Recommendations and Future Works	13

List of Tables

Table No.	Description	Page
1.1	Tools and technologies used	2
2.1	Comparison with related approaches (typical characteristics)	4
3.1	Material families (22 materials, 18 recipe shaders)	6
3.2	The G-buffer (one “data poster” per frame)	7
4.1	Experimental setup	9
4.2	GPU time per pass, 1080p benchmark	9
4.3	Verification summary	10
4.4	Objectives and how they were met	11
6.1	Complex engineering problem attributes	12

List of Figures

Figure No.	Description	Page
1.1	Work plan (days). The single author performed all tasks.	2
3.1	The six layers (dependencies point downward) and the main loop.	4
3.2	From a data table to pixels: content is data, not code.	5
3.3	Noise ingredients (a re-implementation of the shader equations, for illustration): gradient, fractal, domain-warped, ridged, cellular.	5
3.4	One frame, left to right then back along the lower row.	6
3.5	A debugging case. The fog colour sampled the <i>whole</i> sky, stars included, along a horizontal direction, so one star at the horizon painted a vertical streak on every surface in that column (left of centre). The fog now uses the smooth atmosphere only.	7
3.6	Ladder grip state machine (climb 1.1 m/s up, 1.5 m/s down, one rung every 0.3 m).	8
4.1	GPU time per pass (total 10.91 ms of the 16.7 ms budget).	9
4.2	The engine and game in action (screenshots from the program).	10
4.3	Detail results: vegetation, sky and climbing. The painted vehicles are visible in Fig. 3.5b and in the base overview.	10

CHAPTER I

Introduction

1.1 Introduction

Imagine a film studio where nothing is bought: not one prop, painting or costume. Instead, a recipe book tells assistants how to build every tree, wall and cloud while the audience waits, and the same recipe with a different “seed” number gives a different but equally believable forest. *Sentinel* is that studio for a video game. It is a *game engine* (the reusable machinery that draws, simulates and sounds a game world) together with a *game* built on it: a first-person military mission in an open landscape. The word *procedural* means “made by a procedure”: geometry, colours and sounds are calculated by code instead of being stored.

In plain words

A **GPU** is a second processor with thousands of tiny cores, built to colour millions of pixels at once. A **shader** is the small program every core runs for its pixel. A **texture** is a “wallpaper” stretched over a surface. Here, even the wallpapers are painted by shaders at start-up.

1.2 Background and Problem Statement

Computer graphics teaches the pieces separately: transformations, lighting models, texture mapping, noise and image filtering. Real products need them to work *together* at 60 frames per second. Normally the visible content is made in external art tools and loaded as large files, which hides how surfaces and shapes can come from mathematics. The problem addressed here is: *can a complete, good-looking, playable 3D game be produced by code alone, within the limits of a student laptop?* Two limits make this hard. First, rules of the project: no model, image or precomputed data file is allowed. Second, the platform: macOS supports OpenGL up to version 4.1, which has no compute shaders and no shader storage buffers, so every filter and bake must be written as an ordinary full-screen pass.

1.3 Objectives

1. Build a layered engine (platform, GPU, procedural content, world, renderer) with strict downward-only dependencies.
2. Generate all meshes, textures and sounds at run time from code and seeds.
3. Implement a modern real-time renderer: physically based shading, shadows, atmosphere, post-processing.
4. Build a playable game on it: movement, weapons, enemies, vehicles, a helicopter, ladders, doors and a mission.
5. Prove it works: unit tests, GL checks, scripted playthroughs and frame-time measurements within a 60 fps budget.

1.4 Scope

Table 1.1 lists the tools. The scope covers the whole engine and one game; it does not cover networking, multiplayer, hand-made art or console targets.

Table 1.1: Tools and technologies used

Tool	Role (plain words)
Odin language	The programming language of the whole project; fast like C, readable like a script.
OpenGL 4.1 core	The standard way to talk to the graphics card on macOS.
GLSL shaders	Programs run by the GPU for every vertex and pixel (47 files, about 2 000 lines).
GLFW	Opens the window, reads keyboard and mouse.
miniaudio	Sends calculated sound samples to the speakers; all sounds are synthesised.
Git/GitHub, L ^A T _E X/TikZ	Version history (138 commits) and this report.

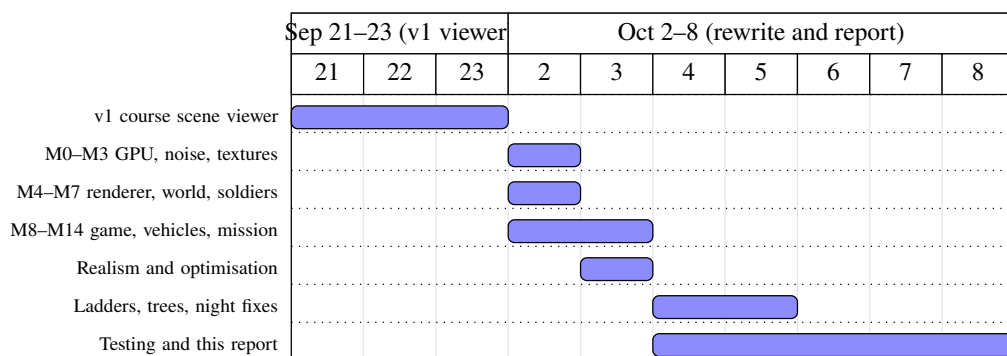


Figure 1.1: Work plan (days). The single author performed all tasks.

1.5 Unfamiliarity of the Problem and its Solution

The design follows published theory but is not copied from a code base. Engines usually import art; demo-scene programs such as *kkrieger* prove that tiny procedural shooters are possible but target a single platform and are not teaching material. Here the constraint “no files” is a *design rule enforced on every feature*, and the engine is written to be read: every file has one feature, every procedure is short, and Docs/Features.md maps each feature to its file. An earlier course version (git tag legacy-v1) was a scene viewer; this report describes the complete rewrite.

1.6 Project Planning and Work Distribution

The work was done by one student, who therefore held every role: designer, programmer, tester, and writer. Figure 1.1 shows the schedule reconstructed from the commit history (138 commits).

1.7 Applications of the Work

The engine is a teaching tool for graphics and procedural generation and a base for small games whose download is only source code. A new material or character is a few lines of data.

1.8 Organization of the Project

Chapter II surveys related work. Chapter III explains the design and every major technique. Chapter IV reports measurements and screenshots. Chapters V and VI discuss social, ethical and engineering-complexity issues, and Chapter VII concludes.

CHAPTER II

Related Works

2.1 Introduction

Sentinel combines four research lines: procedural noise, physically based shading, real-time pipeline tricks and whole procedural games.

2.2 Related Works

Noise. Perlin’s improved noise [1] gives smooth random hills by blending random gradients on a grid; summing several scales (fractal Brownian motion) gives natural detail. Worley’s cellular noise [2] measures distance to random points and yields pebbles, leaves and cracks. Quilez’s domain warping [13] bends the sampling position by another noise to break grid-like patterns. Sentinel uses all three, made *periodic* so textures tile without seams.

Shading. The Cook–Torrance microfacet model [3] with the GGX distribution [4] is the standard physically based reflection model; Karis [5] showed a practical real-time form. Oren–Nayar [8] models rough matte surfaces such as cloth.

Pipeline. Deferred shading [14] separates “what is visible” from “how it is lit”. Cascaded shadow maps [7] give sharp nearby and coarse far shadows. Hillaire’s sky model [6] computes sunsets from scattering physics; ACES tone mapping [9], FXAA [11] and dithered output [10] finish the picture.

Whole games. *kkrieger* (2004) squeezed a shooter into 96 KB by generating content at run time. *Project I.G.I.* (2000) inspired the mission design here: infiltrate a compound, hack, destroy, rescue, extract.

2.3 Discussion

Table 2.1 shows the gap. Existing teaching demos (such as step-by-step OpenGL tutorials) cover one technique each; commercial engines assume imported art; demo-scene shooters are not platform-independent teaching code. Sentinel adds a readable, layered, fully procedural engine with a complete game on macOS-limited OpenGL.

CHAPTER III

Methodology

3.1 Introduction

Making one frame is an assembly line: the game decides what exists, the engine builds its shapes and surfaces, the renderer paints it, and the result appears 60 times per second. This

Table 2.1: Comparison with related approaches (typical characteristics)

Feature	Tutorial demos	Commercial engines	.kkrieger	Sentinel
No imported art	Mixed	No	Yes	Yes
Complete game loop	No	Yes	Yes	Yes
Day/night, clouds, shadows	Rare	Yes	No	Yes
Needs compute shaders	Varies	Often	No	No
Written to be read	Yes	No	No	Yes

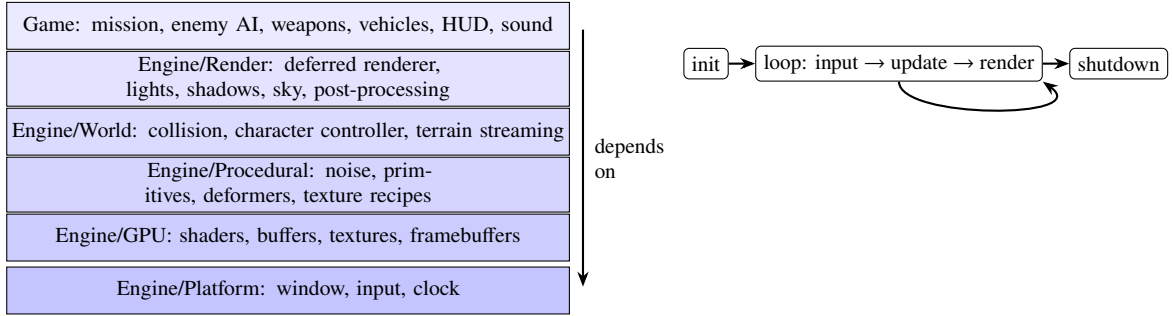


Figure 3.1: The six layers (dependencies point downward) and the main loop.

chapter follows that line.

3.2 Overall Design

Figure 3.1 shows the six layers and the main loop. A lower layer never knows about a higher one, so the renderer can be tested without any game. Source/Main.odin is only a table of contents: initialise, then repeat *input*, *update*, *render*, then shut down. Other rules keep the code readable: procedures at most about 40 lines, full-word names with units last (`range_meters`), assertions on preconditions, and every OpenGL call checked for errors.

3.3 Procedural Geometry and Characters

Every object is a *table* of parts, each a primitive (box, sphere, cylinder, cone, capsule, torus, wedge), a transform, optional deformers and a material (Fig. 3.2). A noise deformer pushes each vertex along its normal $\hat{n}$ by a smooth random amount, so a perfect sphere becomes a lumpy rock; with noise $v \in [-1, 1]$ no vertex moves farther than the amplitude A :

$$\mathbf{p}' = \mathbf{p} + A v(f \mathbf{p}) \hat{n}, \quad \|\mathbf{p}' - \mathbf{p}\| \leq A. \quad (3.1)$$

Taper, bend, twist and bulge are similar one-line deformers. Soldiers are a skeleton of capsules and boxes; a walk is a sine wave of phase, and a leg reaches its foot target by *two-bone inverse kinematics*: for bone lengths a, b and root–target distance c , the knee angle follows from the law of cosines,

$$\cos \theta_{\text{knee}} = \frac{a^2 + b^2 - c^2}{2ab}. \quad (3.2)$$

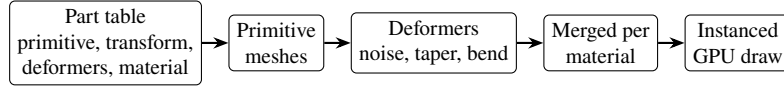


Figure 3.2: From a data table to pixels: content is data, not code.

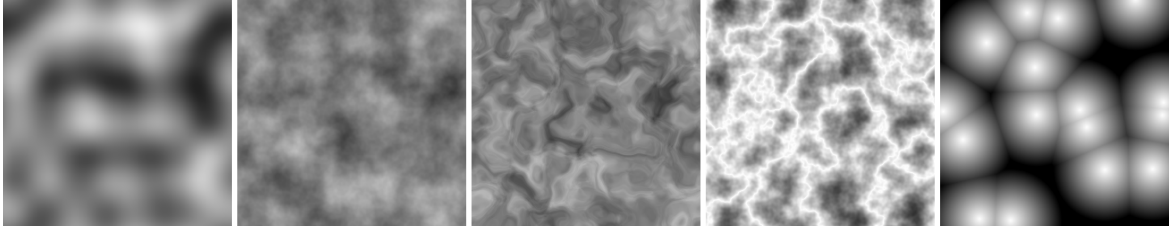


Figure 3.3: Noise ingredients (a re-implementation of the shader equations, for illustration): gradient, fractal, domain-warped, ridged, cellular.

3.4 Procedural Textures

Each material is a *recipe*: a small shader that answers “what is the colour, bumpiness, shininess and crevice-darkness at this point?” The engine runs each recipe once at start-up into a 1024×1024 picture (22 materials from 18 recipes, Table 3.1) and keeps it on the GPU.

In plain words

Gradient noise is like a rubber sheet pinned at grid points, each pinned at a random slope: smooth, never repeating exactly. *Fractal* noise adds finer and finer sheets on top, like hills, boulders and pebbles together.

Gradient noise blends the four corner gradients $\mathbf{g}_c$ of the grid cell with the smooth curve $s(t) = 6t^5 - 15t^4 + 10t^3$, scaled by $\sqrt{2}$ so that values lie in $[-1, 1]$. Fractal noise sums octaves, each at double frequency and half strength:

$$F(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} v(2^i \mathbf{x})}{\sum_{i=0}^{n-1} 2^{-i}}. \quad (3.3)$$

Because corner gradients are chosen from the cell index *modulo the period* P , $F(\mathbf{x} + P\mathbf{e}) = F(\mathbf{x})$: the picture tiles with no seam (checked exactly by the test suite, Section 4.4). Variations give different materials: *domain warping* $F(\mathbf{x} + s[F_1(\mathbf{x}), F_2(\mathbf{x})])$ swirls the pattern (stone, wood, soil); *ridged* noise $\sum a_i(1 - |v_i|)^2$ makes cracks and bark; *cellular* noise gives leaves and pebbles (Fig. 3.3).

Bumps without geometry. A recipe also outputs a height map; the engine turns it into a *normal map*, which fools the lighting into seeing bumps. The height slope uses the Scharr 3×3 filter, the most direction-fair small filter:

$$\frac{\partial h}{\partial x} \approx \frac{3(h_{NE} + h_{SE} - h_{NW} - h_{SW}) + 10(h_E - h_W)}{32 \Delta}, \quad \hat{n} = \frac{(-k h_x, -k h_y, 1)}{\|\cdot\|}. \quad (3.4)$$

Where the height is below its neighbours’ average (a crevice), albedo and ambient occlusion are darkened. On objects the picture is applied by *triplanar mapping*: it is projected along the three axes and blended by $w_i = |n_i|^k / \sum_j |n_j|^k$, so deformed shapes never stretch the texture.

Table 3.1: Material families (22 materials, 18 recipe shaders)

Family	Recipe idea
Concrete, rock, sand, dirt, asphalt	Warped fractal tone, pores/pebbles from cellular noise, ridged cracks
Painted, rusted, gun metal; olive paint	Panel seams, scratches, sun fade, rust patches with pits and streaks
Wood, bark, birch bark	Stretched ridged noise (fissures), sine rings under fibres
Leaves, needles, grass	One cell per leaf/needle/blade tip, tone from the cell's random roll
Canvas, camo, fabrics, rubber, skin, glass	Woven threads, ripstop grid, grooves, pores, smooth glass

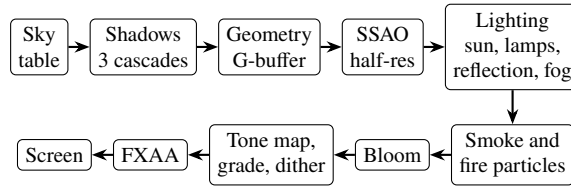


Figure 3.4: One frame, left to right then back along the lower row.

3.5 The Rendering Pipeline

In plain words

Deferred shading is like photographing a “data poster” of the scene first (every pixel’s colour, direction and shininess), then lighting the poster once instead of lighting every object separately. It makes many lights cheap.

The poster is the *G-buffer* (Table 3.2). Figure 3.4 lists the passes. Direct light uses the Cook–Torrance model [3]: a diffuse part plus a mirror-like specular part,

$$f_s = \frac{D F G}{4(\mathbf{n} \cdot \mathbf{l})(\mathbf{n} \cdot \mathbf{v})}, \quad D = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{h})^2(\alpha^2 - 1) + 1)^2}, \quad F = F_0 + (1 - F_0)(1 - \mathbf{v} \cdot \mathbf{h})^5, \quad (3.5)$$

where D says how many tiny mirror facets face the viewer (GGX), G how much they shadow each other, F how much light reflects at a glancing angle (Schlick), and α is the surface roughness squared. Six illumination models are selectable per pixel (Lambert, Phong, Blinn–Phong, Oren–Nayar, Cook–Torrance, subsurface). Distant things fade into haze: with distance d and density $k = 3.5 \times 10^{-4} \text{ m}^{-1}$,

$$C = T C_{\text{surface}} + (1 - T) C_{\text{haze}}, \quad T = e^{-kd}. \quad (3.6)$$

Finally the HDR image is compressed to screen range with the ACES curve [9] (Narkowicz fit), graded, dithered [10] and smoothed (FXAA [11]):

$$C_{\text{out}} = \text{clamp}\left(\frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}\right), \quad x = \text{exposure} \times \text{radiance}. \quad (3.7)$$

Table 3.2: The G-buffer (one “data poster” per frame)

Layer	Format	Contents
Albedo + model id	sRGB 8-bit	Surface colour; which illumination model
Normal + roughness + metal	16-bit float	Direction packed in two numbers (octahedral), shininess
Emission + occlusion	16-bit float	Light the surface gives off; crevice darkness
Depth	32-bit float	Distance; world position is rebuilt from it



(a) Before: bright vertical line



(b) After: fixed

Figure 3.5: A debugging case. The fog colour sampled the *whole* sky, stars included, along a horizontal direction, so one star at the horizon painted a vertical streak on every surface in that column (left of centre). The fog now uses the smooth atmosphere only.

3.6 Sky, Time of Day and Clouds

The sun’s direction comes from astronomy: with latitude φ , declination δ and hour angle $h = 15^\circ(\text{hours} - 12)$, its height is

$$\sin(\text{elevation}) = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos h. \quad (3.8)$$

The colour of the sky is precomputed each frame into a small table from Rayleigh and Mie scattering (blue sky, red sunset) [6]; the moon and a hashed star field take over at night. Clouds live on two imaginary planes, cumulus at $H = 1500$ m and thin cirrus at 7 km. A view ray with direction $\mathbf{d}$ meets the plane at

$$\mathbf{P}_{xz} = \frac{H}{d_y} (d_x, d_z) + \mathbf{w} t, \quad (3.9)$$

where $\mathbf{w}$ is the wind velocity: shifting the noise by $\mathbf{w}t$ makes clouds drift, and a faster-moving detail layer lets their shapes slowly change. The cloud colour is the sky’s own grey, plus a sun rim by day and a moon rim at night.

3.7 World, Physics and Gameplay

Terrain. Ground height is fractal noise, warped and blended with a rounded ridged term, flattened into a plateau for the base: $h = h_0 + A \nu(x, z)$ smoothstep($r_0, r_0 + w, \|\mathbf{p} - \mathbf{c}\|$). It is meshed in 64 m chunks that stream around the player; layers (grass, dirt, rock, sand) blend by slope and height with wandering borders.

Trees. Four species (oak, pine, birch, cypress) are each a few tables of limbs and leaf lumps.

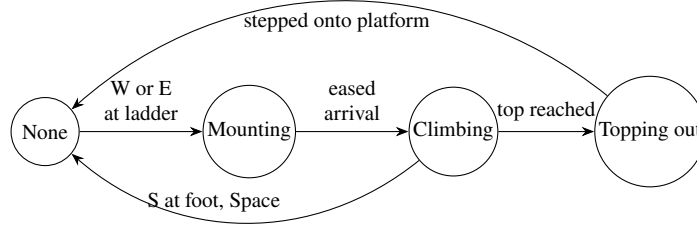


Figure 3.6: Ladder grip state machine (climb 1.1 m/s up, 1.5 m/s down, one rung every 0.3 m).

Every instance gets its own height, width, lean and tint, and the vertex shader bends it as a cantilever, $\Delta_{xz} \propto (0.04 + 0.1g)(y/9)^2$, where $g = \frac{1}{2} + \frac{1}{2} \sin(1.3t + 0.05x + 0.03z)$ is a travelling gust, plus a small leaf flutter.

Collision and vehicles. Solids are boxes that may be rotated, so walls are exact at any angle; the player is a capsule. Cars use the bicycle model, $\dot{\psi} = v \tan \delta / L$ (turn rate from speed, steering angle and wheel base L), and engine power fades toward top speed so acceleration is strong from rest and eases off:

$$\dot{v} = a_0 \left(1 - 0.6 \frac{|v|}{v_{\max}} \right). \quad (3.10)$$

Four ground vehicles and a helicopter are boarded with E; the helicopter has a rotor spin-up and arcade flight model.

Ladders are a four-state machine (Fig. 3.6); while gripping, the controller, not the collision solver, owns the body, which removes getting stuck. Hands grip rungs hand over hand.

Mission, AI and sound. A staged objective machine drives five goals: enter the compound, hack the HQ computer, destroy the radar, rescue the hostage, reach the beacon. Soldiers patrol, see, hear (crouching is quieter), raise an alarm and call reinforcements; cameras sweep. All sounds (shots, steps, engines, alarm) are synthesised by oscillators and filtered noise shaped by envelopes and mixed in code. Progress is saved as plain text.

3.8 Conclusion

Each technique is small and data-driven; together they give a complete, deterministic world from seeds.

CHAPTER IV

Implementation, Results and Discussions

4.1 Introduction

This chapter states how the work was measured and shows what it produces.

4.2 Experimental Setup

Table 4.1 gives the environment. Frame times come from the built-in benchmark (a scripted 1 km flight at 1080p, 300 frames, release build).

Table 4.1: Experimental setup

Item	Value
Machine	Apple-silicon Mac, macOS, OpenGL 4.1 core profile
Compiler / flags	Odin dev-2026-09, <code>-o:speed</code>
Code size	164 Odin files (about 19 200 lines), 47 GLSL files (about 2 000 lines)
Display / benchmark	2560×1440 screenshots; 1920×1080 benchmark

Table 4.2: GPU time per pass, 1080p benchmark

Pass	Time (ms)	Share
Geometry (G-buffer)	4.11	38%
Lighting (sun, ambient, reflections, fog)	2.79	26%
Shadows (3 cascades)	1.58	14%
Post-processing	1.21	11%
Ambient occlusion	1.20	11%
Sky table	0.02	0%
GPU total	10.91	100%

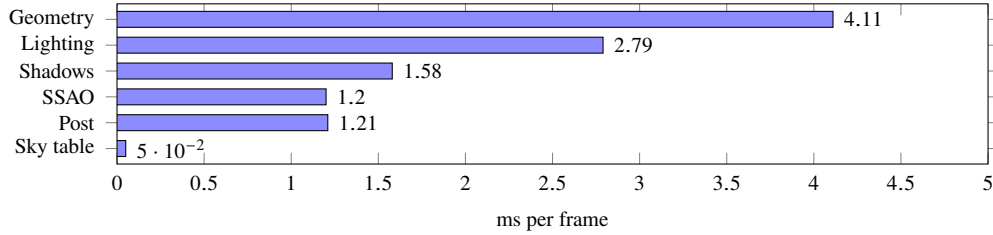


Figure 4.1: GPU time per pass (total 10.91 ms of the 16.7 ms budget).

4.3 Evaluation Metrics

Speed: frame time $t = 1000/\text{fps}$ ms (budget 16.7 ms for 60 fps) and GPU time per pass.

Correctness: pass/fail of unit tests, GL checks and a scripted playthrough that must complete five objectives per mission.

Tiling error: $\max |f(\mathbf{u} + \mathbf{k}) - f(\mathbf{u})|$ for integer $\mathbf{k}$, which must be exactly zero for every material.

Dataset: none; all content comes from fixed seeds, so every run is reproducible.

4.4 Implementation and Results

4.4.1 Quantitative results

The average frame was 16.75 ms because the display synchronises to 60 Hz; the GPU work itself needs 10.9 ms (Table 4.2, Fig. 4.1), leaving headroom. Table 4.3 lists the verification: 281 unit tests, 30 + 8 318 + 318 567 GL checks, and both missions complete in 5 of 5 objectives.

4.4.2 Qualitative results

Figures 4.2 and 4.3 show the running game. The showroom (Fig. 4.2c) is an automatic gallery with one sphere per material, so any new recipe is seen at once. Figure 4.3 shows tree species,

Table 4.3: Verification summary

Suite	Count	What it checks
Unit tests (13 packages)	281	Noise, meshes, IK, collision, ladders, vehicles, mission, audio
TextureCheck	8 318	Exact tiling of all materials, normals, colour round trip
RenderCheck	318 567	Lighting models, shadows, atmosphere, instancing
GpuCheck	30	Textures, framebuffers, shaders
Scripted playthrough	2	Both missions: 5 of 5 objectives

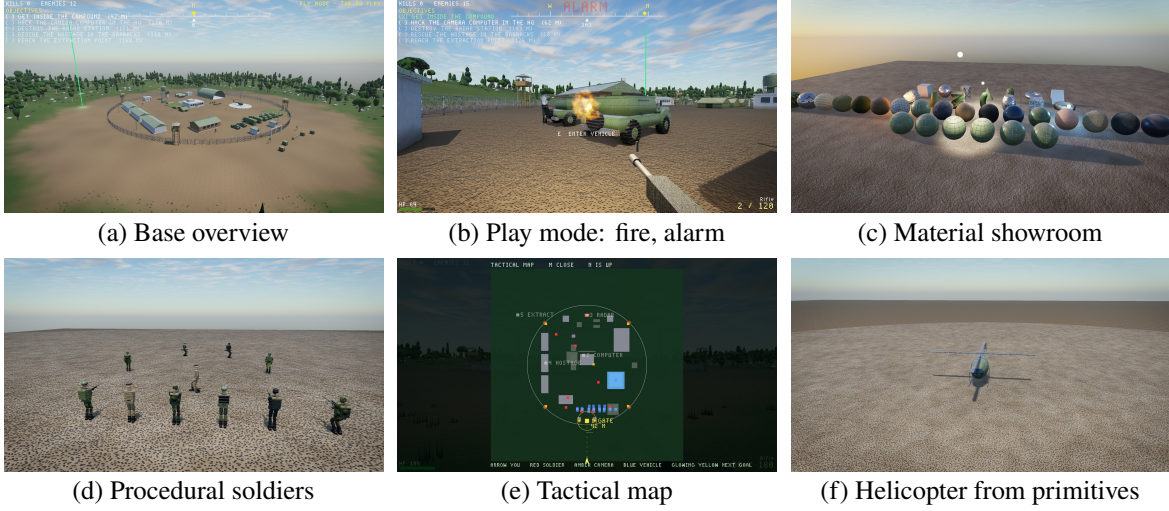


Figure 4.2: The engine and game in action (screenshots from the program).

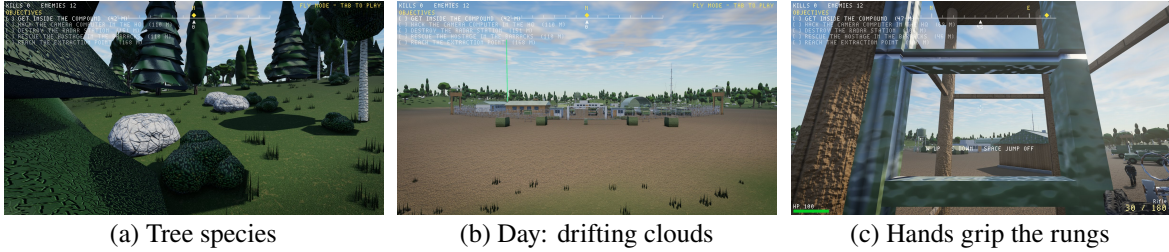


Figure 4.3: Detail results: vegetation, sky and climbing. The painted vehicles are visible in Fig. 3.5b and in the base overview.

drifting clouds and hands on a ladder; the night scene with moonlit clouds is Fig. 3.5b.

4.4.3 Analysis

Geometry is the largest cost because trees, base and terrain are drawn in detail, so instancing (one draw call for thousands of trees) and per-chunk culling matter most. Cheaper clouds (no warp in the sun-shading march, fewer octaves) kept the sky from growing the lighting pass. The bug in Fig. 3.5 was found by bisecting: remove draw items, lights and effects until the line disappeared, which pointed to the fog; it shows why every pass must have a test and a screenshot.

4.5 Objectives Achieved

Table 4.4 compares the five objectives of Section 1.3 with the results.

Table 4.4: Objectives and how they were met

Objective	Result
1 Layered engine	Six layers, dependencies downward only; <code>Main.odin</code> is a table of contents.
2 All content from code	No model, image or sound file; 22 materials, 4 tree species, 5 vehicles, soldiers, synthesised audio.
3 Modern renderer	Deferred PBR, cascaded shadows, SSAO, SSR, sky, clouds, bloom, ACES, FXAA.
4 Playable game	Two missions, doors, ladders, vehicles, helicopter, stealth, saves.
5 Verified	281 tests, 327 000 GL checks, 2 playthroughs, 10.9 ms GPU time.

4.6 Conclusion

The measurements show that a fully procedural engine can meet a 60 fps budget on a laptop while passing exact correctness checks.

CHAPTER V

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

All code, shapes, textures and sounds are original; there are no third-party assets to license. Libraries (Odin, GLFW, miniaudio) are open source under permissive licences. The mission is inspired by, not copied from, *Project I.G.I.*; no names, maps or art are reused.

5.2 Ethical Considerations

The game is a fiction about an invented compound with abstract, low-detail figures; it contains no real places, groups or people. It is intended for adult audiences. No personal data is collected: saves are local plain-text checkpoints.

5.3 Safety Considerations

Muzzle flash, alarm and explosions can bother light-sensitive players, and loud sound can hurt hearing, so brightness changes are smoothed, flashes are short and volume is mixed and limited. Playing sessions should include breaks.

5.4 Legal Considerations

Open-source licence terms are respected. Violence is stylised and the project makes no claim to real military accuracy. The report credits all cited work.

Table 6.1: Complex engineering problem attributes

Attribute	In this project
Depth of knowledge	Radiometry, noise theory, numerical filters, GPU pipelines, collision, state machines, synthesis.
Conflicting requirements	Realism versus 16.7 ms; no files versus rich surfaces; OpenGL 4.1 without compute.
Depth of analysis	Debugging the fog star streak; deriving tiling guarantees; tuning physics curves.
Interdependence	Renderer, world, game and audio share data layouts and timing.

5.5 Impact on Societal, Health and Cultural Issues

Positive: an open, readable engine lowers the barrier to learning graphics. Negative: shooter games may encourage long play; a “peaceful” rescue and stealth emphasis (crouching, avoiding detection) is part of the design.

5.6 Impact on the Environment and Sustainability

Because no art files are downloaded, the whole project is a few megabytes of source, reducing storage and transfer energy. Frame-rate limiting through display synchronisation avoids needless GPU work.

CHAPTER VI

Addressing Complex Engineering Problems and Activities

6.1 Complex Engineering Problems

Table 6.1 maps the project to the usual attributes.

6.2 Complex Engineering Activities

The project used modern tools (compiler, profiler, test runners), integrated many components and verified quality by measurement.

CHAPTER VII

Conclusions

7.1 Summary

Sentinel shows that a complete 3D game, from surfaces and shapes to weather and sound, can be generated entirely by code under an OpenGL 4.1 limit. A six-layer engine, a deferred physically based renderer, noise-based material recipes, table-driven objects, a mission game with vehicles, ladders and stealth, and an extensive test suite fit in about 21 000 lines and run within a 60 fps budget.

7.2 Limitations

Models are blocky (primitives plus noise); there is no true global illumination or ray tracing; foliage is lumpy volumes rather than individual leaves; the beacon is a solid column; guards cannot pick up dropped weapons; and the game was tuned by scripts and screenshots rather than a large playtest.

7.3 Recommendations and Future Works

Add leaf cards with cut-out transparency, screen-space global illumination, point-light shadows, anisotropic highlights, networked multiplayer, and a larger user study; port to a modern graphics API with compute shaders for GPU-side generation.

REFERENCES

- [1] K. Perlin, “Improving noise,” *ACM Trans. Graph. (SIGGRAPH)*, vol. 21, no. 3, pp. 681–682, 2002.
- [2] S. Worley, “A cellular texture basis function,” in *Proc. SIGGRAPH*, 1996, pp. 291–294.
- [3] R. L. Cook and K. E. Torrance, “A reflectance model for computer graphics,” *ACM Trans. Graph.*, vol. 1, no. 1, pp. 7–24, 1982.
- [4] B. Walter, S. R. Marschner, H. Li and K. E. Torrance, “Microfacet models for refraction through rough surfaces,” in *Proc. Eurographics Symp. Rendering*, 2007, pp. 195–206.
- [5] B. Karis, “Real shading in Unreal Engine 4,” SIGGRAPH Course: Physically Based Shading in Theory and Practice, 2013.
- [6] S. Hillaire, “A scalable and production ready sky and atmosphere rendering technique,” *Computer Graphics Forum*, vol. 39, no. 4, pp. 13–22, 2020.
- [7] R. Dimitrov, “Cascaded shadow maps,” NVIDIA White Paper, 2007.
- [8] M. Oren and S. K. Nayar, “Generalization of Lambert’s reflectance model,” in *Proc. SIGGRAPH*, 1994, pp. 239–246.
- [9] K. Narkowicz, “ACES filmic tone mapping curve,” blog post, 2016.
- [10] J. Jimenez, “Next generation post processing in Call of Duty: Advanced Warfare,” SIGGRAPH Course: Advances in Real-Time Rendering, 2014.
- [11] T. Lottes, “FXAA,” NVIDIA White Paper, 2009.
- [12] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 4.1*, Khronos Group, 2010.
- [13] I. Quilez, “Domain warping,” online article, iquilezles.org.
- [14] S. Hargreaves and M. Harris, “Deferred shading,” NVIDIA presentation, Game Developers Conference, 2004.
- [15] H. Scharr, “Optimal operators in digital image processing,” Ph.D. dissertation, Univ. Heidelberg, 2000.
- [16] T. Akenine-Möller, E. Haines and N. Hoffman, *Real-Time Rendering*, 4th ed., CRC Press, 2018.